N16 Principles of Machine Learning

Bohaz

19.09.2026

Contents

I Supervised learning **2**

1 Linear regression **2**

2 Non-linear regression **6**

3 Classification **9**

II Unsupervised learning **14**

4 Dimensionally reduction **14**

5 Clustering **18**

Part I
Supervised learning

1 Linear regression

In school, quite often, we are asked to fit a straight line of the form $y = mx+c$ though a set of data points $\{x_i, y_i\}$, For example, y_i is the forces exerted by by some iron weights and x_i is the displacement due to those weights on a spring, and you are asked to determine the stiffness of the spring k. In other words, we hypothesis that data comes from the underlying model, which is Hookes' law

$$F = -kx \tag{1.1}$$

and want to infer k the stiffness of the spring from the experimental data $\{x_i, y_i\}$ in N columns $i = 1, 2, \ldots, N$, which are pairs of displacements of the spring and the weight you put on it. we expected our observed data to also be subjected to random noise ξ_i, such as a small part of the iron disk is chipped off, despite the disk still has the the label 100 g engraved on it, or it is oxidised, and some oxygen has combined with the iron and formed a bit of rust and caused the weight to increase slightly, so we expect our data to have the form

$$y_i = kx_i + \xi_i \tag{1.2}$$

the minus sign is gone because the force in Hookes' law is a restoring force, whereas the weights we put on the spring, is a stretching force, i.e. not restoring. The point of us fitting a straight line is to determine the slope of the line k which is also the stiffness of the spring. And we are typically asked by teachers to minimise the following summed squared errors

$$L(k) = \sum_i (y_i - kx_i)^2 \tag{1.3}$$

with respect to k. However, when I asked the teachers why we should minimise mean square but not some other quantity, non of the answers I got was satisfactory, and the goal of this chapter is to answer this question properly, using basic probability theory.

Now consider two events A and B the probability both happening is the joint probability

$$P(A, B) = P(A|B)P(B) = P(B|A)P(A) \tag{1.4}$$

and when A and B are independent event $P(A, B) = P(A)P(B)$ but we should not expect this to be true in general. Rearanging yields,

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \tag{1.5}$$

this simple formula is known as Bayes' theorem, it is very useful, because typically, in real life, we do not know A, in this case, is the stiffness of the spring stiffness k, but we know B, which is $\{x_i, y_i\}$, which written down in our lab notebook in N columns, and we want to know k. We now introduce some on, which will be handy when communicating with professionals in this field, $P(A|B)$ is known as the **posterior**, $P(B|A)$ is known as the **likelihood**, $P(A)$ is known as the **prior** and $P(B)$ is known as the **evidence**. There is a

lot braces around, so to clean up notation, we introduce $\mathbf{x} = \{x_i\}$, $\mathbf{y} = \{y_i\}$ and $\boldsymbol{\xi} = \{\xi_i\}$.

$$P(k|\mathbf{x},\mathbf{y}) = \frac{P(\mathbf{x},\mathbf{y}|k)P(k)}{P(\mathbf{x},\mathbf{y})} \tag{1.6}$$

Now consider

$$P(A,B,C) = P(A,B|C)P(C) = P(B|A,C)P(A,C) \tag{1.7}$$

which is a useful rule for manipulating probability, when we move A from LHS to RHS in the conditional probability, A must also pop up in $P(C)$. Using this rule to the equation above gives

$$P(k|\mathbf{x},\mathbf{y}) = \frac{P(\mathbf{y}|k,\mathbf{x})P(k,\mathbf{x})}{P(\mathbf{x},\mathbf{y})} \tag{1.8}$$

we should put our hypothesis that the spring follows Hookes' law in all the right arguments of the probabilities, we will not do this to simplify notation, but it should be left as understood. Now use $P(k,\mathbf{x}) = P(k|\mathbf{x})P(\mathbf{x})$ and $P(\mathbf{x},\mathbf{y}) = P(\mathbf{y}|\mathbf{x})P(\mathbf{x})$ then

$$P(k|\mathbf{x},\mathbf{y}) = \frac{P(\mathbf{y}|k,\mathbf{x})P(k|\mathbf{x})P(\mathbf{x})}{P(\mathbf{y}|\mathbf{x})P(\mathbf{x})} \tag{1.9}$$

cancelling the common $P(\mathbf{x})$ yields

$$P(k|\mathbf{x},\mathbf{y}) = \frac{P(\mathbf{y}|k,\mathbf{x})P(k|\mathbf{x})}{P(\mathbf{y}|\mathbf{x})} \tag{1.10}$$

But note that $P(\mathbf{y}|k,\mathbf{x}) = P(\boldsymbol{\xi}|k,\mathbf{x})$ because if we know the noise $\boldsymbol{\xi} = \{\xi_i\}$, $\mathbf{x}$ and k, we definitely know $\mathbf{y}$ with zero uncertainty from Eq.(2). So

$$P(k|\mathbf{x},\mathbf{y}) = \frac{P(\boldsymbol{\xi}|k,\mathbf{x})P(k|\mathbf{x})}{P(\mathbf{y}|\mathbf{x})} \tag{1.11}$$

now if we believe the noise is independent of k and $\mathbf{x}$ (i.e. in our context, how much the iron disk is chipped or oxidised does not depend on the stiffness of the spring or displacement of the spring) and uncorrelated, meaning each measurement i you get is not dependent on what you got previoiusly, we have

$$P(\boldsymbol{\xi}|k,\mathbf{x}) = P(\boldsymbol{\xi}) = \prod_i P(\xi_i) \tag{1.12}$$

then if we also believe that the noise if normally distributed with some standard deviation σ, then

$$P(\xi_i) = \frac{1}{\sqrt{2\pi\Delta^2}} e^{-\xi_i^2/2\sigma^2} \tag{1.13}$$

and

$$P(\boldsymbol{\xi}) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\xi_i^2/2\sigma^2} = \frac{1}{(2\pi\sigma^2)^{N/2}} e^{-(\sum_i \xi_i^2)/2\sigma^2} \tag{1.14}$$

and if the spring actually obeys Hookes' law, then its stiffness is a property of the spring, and should not depend on its displacement, so $P(k|\mathbf{x}) = P(k)$, then substitute back into Eq.(11) yields

$$P(k|\mathbf{x},\mathbf{y}) = Ce^{-\sum_i \xi_i^2/2\sigma^2} P(k) = Ce^{-\sum_i \xi_i^2/2\sigma^2} e^{\ln P(k)} \tag{1.15}$$

where $C = 1/P(\mathbf{y}|\mathbf{x})(2\pi\Delta^2)^{N/2}$ is a normalisation constant to ensure $\int dk\, P(k|\mathbf{x}) = 1$, nothing of interest. We now use $\xi_i = y_i - kx_i$, then, maximising the posterior $P(k|\mathbf{x}, \mathbf{y})$ is the same as minimising the following

$$L(k) = \sum_i (y_i - kx_i)^2 - 2\sigma^2 \ln P(k) \tag{1.16}$$

if we assume that the iron disks are not too absurdly chipped, like half of it is gone, (teacher would not let us use such a disk) or way too rusty, we expect σ^2 to be small enough to be negligible, then inferring the stiffness of the spring from fitting a straight line through the data points by minimising the square error Eq.(3)

$$L(k) = \sum_i (y_i - kx_i)^2 \tag{1.17}$$

is same as maximising the posterior $P(k|\mathbf{x}, \mathbf{y})$. Given our assumption, 1. The spring obeys Hookes' law. 2. Independent, 3. uncorrelated 4. Gaussian distributed 5. small noise ξ_i. Which is what the teachers really meant when they told us to do a least square fit, only at the time, we were too young at the time to understand why, now we are grown up and understand the reason behind it.

Here, we have considered only one parameter (i.e. the slope k), in general, a model have many parameter, so the generalisation is $k \to \mathbf{w}$ where $\mathbf{w} = \{w_j\}_{j=1}^M$ and the square loss becomes

$$L(\mathbf{w}) = \sum_{i=1}^{N} \left(y_i - \sum_{j=1}^{M} x_{ij} w_j \right)^2 = |\mathbf{y} - \mathbf{X}\mathbf{w}|^2 \tag{1.18}$$

$L(\mathbf{w})$ more generally known as the **loss function**. It is made of both an error function, in this case $|\mathbf{y} - \mathbf{X}\mathbf{w}|^2$, which can also be understood as log likelihood, and the square error is just one type of it, for a Gaussian likelihood. The second part is known as regulariser, which we will talk about in a bit.

The best $\mathbf{w}$ should minimise the loss function. $\mathbf{w}$ is known as the **weights**, and the progress of doing a curve fitting (although high dimension in this case) is same as finding out the best weights, and this is more generally known as **learning**. A technical aspect is that the weights should live in parameter, and be represented as row vectors $\mathbf{w}^T$, which is not the same space that data $\mathbf{x}, \mathbf{y}$ live in, we will not stress ourselves with this too much, but should at least be aware of it. The second part of a loss function is on RHS of Eq.(16), the log of the prior distribution of the parameters is know as the **regularisor**. Conversely, choosing a regularisor for a loss function is same as choosing a prior belief. Let us see an example to illustrate this: if we have the prior belief that

$$P(\mathbf{w}) = \prod_j \frac{1}{\sqrt{2\pi\Delta^2}} e^{-w_j^2/2\Delta^2} \tag{1.19}$$

then we have $e^{\ln P(\mathbf{w})} \propto e^{\sum_j w_j^2/2\Delta^2}$ at the end of Eq.(15), then Eq.(16) becomes

$$L(\mathbf{w}) = |\mathbf{y} - \mathbf{X}\mathbf{w}|^2 - \frac{\sigma^2}{\Delta^2}|\mathbf{w}|^2 \tag{1.20}$$

where $|\mathbf{w}|^2$ is the regularizer and $\lambda = \sigma^2/\Delta^2 > 0$ is known as the regularisation strength, it punishes $|\mathbf{w}|^2$ being too large (springs being too stiff), and it can be understood as a

ratio of the characteristic widths of your prior believes of parameters to the characteristic width of the likelihood.

So in summary: The loss function is made of two parts: An error function, which is the minus log likelihood, and a regularisor, which is the log prior. The regularisor, is a scale factor in front of the regularisor, then depends on the characteristic width of the likihood and prior. The loss function is an inference of parameters given data.

We have considered a simple example to illustrate one realisation of the concepts, but machine learning is general: Given data $\{x_i, y_i\}$, we want to find the underlying model, which is characterised by $\mathbf{w}$, that is responsible to generating these data. We want to infer the ground truth of nature, based on our observation of the manifestation and consequences of the ground truth.

Lets now look at how we can actually find these weights practically. We now use new notation for error function

$$C(\mathbf{w}) = \frac{1}{2}|\mathbf{y} - \mathbf{X}\mathbf{w}|^2 = \frac{1}{2}\sum_i \left(y_i - \sum_j X_{ij} w_j \right)^2 \tag{1.21}$$

we wish to adjust the weights to minimise the error, so we differentiate then set the derivative to zero

$$\begin{aligned}
\frac{\partial C}{\partial w_l} &= -\sum_i \left(y_i - \sum_j X_{ij} w_j \right) \frac{\partial}{\partial w_l} \sum_k X_{ik} w_k \\
&= -\sum_i \left(y_i - \sum_j X_{ij} w_j \right) \sum_k X_{ik} \frac{\partial w_k}{\partial w_l} \\
&= -\sum_i \left(y_i - \sum_j X_{ij} w_j \right) \sum_k X_{ik} \delta_{kl} \\
&= -\sum_i \left(y_i - \sum_j X_{ij} w_j \right) X_{il} \\
&= -\sum_i X^T_{li} \left(y_i - \sum_j X_{ij} w_j \right)
\end{aligned} \tag{1.22}$$

set to zero, the write in vector notation

$$\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}^*) = 0 \tag{1.23}$$

rearrange yields the solution to the optimal weights

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \equiv \mathbf{X}^+\mathbf{y} \tag{1.24}$$

where we have defined pseudo inverse $\mathbf{X}^+$. Repeating the analysis with the regularisor

$$L(\mathbf{w}) = \frac{1}{2}|\mathbf{y} - \mathbf{X}\mathbf{w}|^2 + \frac{1}{2}\lambda|\mathbf{w}|^2 \tag{1.25}$$

yields

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{1})^{-1}\mathbf{X}^T\mathbf{y} \tag{1.26}$$

once the weights has been obtained, we have a model that can be used for prediction for unseen input.

2 Non-linear regression

a

In the previous section, we looked at linear least square in one dimension (finding out the stiffness of the spring), then we looked at linear least least square in high dimension. In this chapter, we are interested in solving the non-linear least square problem. i.e. We wish to minimise the cost/error function

$$C(\mathbf{z}) = \frac{1}{2}\sum_i r_i(\mathbf{z})^2 = \frac{1}{2}|\mathbf{r}(\mathbf{z})|^2 \tag{2.1}$$

where $\mathbf{z}$ is a set of adjustable parameters, and r_i are errors between the ground truth and our model prediction, we could also call r_i the residue. Non linear just means r_i is not a linear function of z_k. Define the derivative of error with respect to parameter as Jacobian

$$J_{ik} \equiv \frac{\partial r_i}{\partial z_k} \tag{2.2}$$

then one can show

$$\frac{\partial C}{\partial z_k} = \sum_i r_i J_{ik} = \sum_i J_{ki}^T r_i \tag{2.3}$$

in vector notation

$$\mathbf{g} = \nabla C = \mathbf{J}^T\mathbf{r} \tag{2.4}$$

our goal is to adjust all the z_k until we minimise C. The basic idea is that we start with some $\mathbf{z}$, then iteratively update $\mathbf{z} \leftarrow \mathbf{z} + \delta\mathbf{z}$ until we reach a minimum in C. The simplest way to do this is just move $\mathbf{z}$ in the direction of max negative slope of the cost function

$$\delta\mathbf{z} = -\alpha\mathbf{g} \tag{2.5}$$

where $\alpha > 0$ controls the step size, We are done if $\delta\mathbf{z}$ is small enough. This method is known as **gradient descent**, it is quite simple but only uses the local slope. In other words, how do we choose a correct α? The slope does not tell us how far to move before the slope changes substantially. To illustrate what we mean by this, consider $C(z_1, z_2) = \frac{1}{2}(100z_1 + z_2)^2$, which has minimum $z_1 = z_2 = 0$ and slope is $\nabla C = (100z_1, z_2)$ and if we are currently at $(z_1 = 0.01, z_2 = 1)$, then $\nabla C = (1, 1)$, now we are in trouble if we use gradient descent, because setting $\alpha = 1$, will give $z_2 = 0$ (good), $z_1 = -0.99$ (bad, we massively overshot) and if we set $\alpha = 0.01$ then $z_1 = 0$ (good), but $z_2 = 0.99$ (bad, it barely changed and has got a long way to go before reaching the minimum $z_2 = 0$), what is missing is the curvature information. We need α appropriate to each z_k based on the curvature information.

Gauss-Newton uses more information, for small $\delta\mathbf{z}$ it approximates

$$r_i(\mathbf{z} + \delta\mathbf{z}) \simeq r_i(\mathbf{z}) + \sum_k \frac{\partial r_i}{\partial z_k}\delta z_k \tag{2.6}$$

in vector notation

$$\mathbf{r}(\mathbf{z} + \delta\mathbf{z}) \simeq \mathbf{r}(\mathbf{z}) + \mathbf{J}\delta\mathbf{z} \tag{2.7}$$

then it chooses a step that minimises $\frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2$, i.e. find the new step $\delta\mathbf{z}$ so the $\mathbf{z} + \delta\mathbf{z}$ gives lowest cost compared with others, differentiating $\frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2$, with respect to z_k then

using local linearity approximation Eq.(7) and settting to zero gives

$$\sum_i \left(r_i + \sum_k J_{ik} \delta z_k \right) J_{il} = 0 \tag{2.8}$$

in vector notation

$$\mathbf{J}^T(\mathbf{r} + \mathbf{J}\delta\mathbf{z}) = 0 \tag{2.9}$$

then rearrange yields

$$\mathbf{J}^T\mathbf{J}\delta\mathbf{z} = -\mathbf{J}^T\mathbf{r} = -\mathbf{g} \tag{2.10}$$

$\mathbf{J}^T\mathbf{J}$ is positive semi-definite because for some vector $\mathbf{v}$ we have $\mathbf{v}^T\mathbf{J}^T\mathbf{J}\mathbf{v} = |\mathbf{J}\mathbf{v}|^2 \geq 0$. To prevent various problems the arises $\mathbf{J}^T\mathbf{J}$ have eigenvalues as zeros, which can be numerically nasty (Such as solution not being unique and/or will result in dividing by zeros in the algorithm) we add a damping term, to guarantee $\mathbf{J}^T\mathbf{J}$ positive definite. This is known as **Levenberg-Marquardt**

$$(\mathbf{J}^T\mathbf{J} + \lambda\mathbf{1})\delta\mathbf{z} = -\mathbf{g} \tag{2.11}$$

where $\lambda > 0$ to ensure positive definiteness and non-singularity, so we can refactorise it into a lower triangular matrix $\mathbf{L}$ and its transpose using Cholesky factorisation as explained in P2 and P3

$$\mathbf{J}^T\mathbf{J} + \lambda\mathbf{D} = \mathbf{L}\mathbf{L}^T \tag{2.12}$$

then $\delta\mathbf{z}$ can be solved by doing a forward substitution followed by a back substitution. In the limit of weak damping $\lambda \to 0$, we recover Gauss-Newton method.

b

Three new problems arises due to the somewhat adhoc/arbitrary parameter λ, which we should feel just as uncomfortable as the original step size α, that we introduced to ensure the positive definiteness of the matrix that controls adaptive step size in gradient descent, so our Cholesky factorisation and solve can work as intended. The two problems are: 1. We just added the $\lambda\mathbf{1}$ term in Eq.(37) without deriving it, where does it come from, and 2. If λ is small, then $|\delta\mathbf{z}|$ is large, and will it cause the local linear approximation Eq.(33) to break down. So we look at these problems in this section.

Now consider minimising the the cost $\frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2$ under the constraint that step size is small,

$$|\delta\mathbf{z}|^2 \leq \rho^2 \tag{2.13}$$

where $\rho > 0$ controls the permitted step size to ensure approximation in Eq.(7) is satisfied. This can be done using Legrange's undetermined multiplier method with multiplier λ, however, it differs from usual Legrange multiplier, because Eq.(14) is an inequality. The Legrangian is

$$\mathcal{L}(\delta\mathbf{z}) = \frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2 + \frac{1}{2}\lambda\left(|\delta\mathbf{z}|^2 - \rho^2\right) \tag{2.14}$$

if $|\delta\mathbf{z}|^2 < \rho^2$, we do not need the constraint so $\lambda = 0$ in Eq.(41). But if $|\delta\mathbf{z}|^2 = \rho^2$, then this reduces back to a normal Lagrange undetermined multiplier method. But notice that in both cases, we have

$$\lambda\left(|\delta\mathbf{z}|^2 - \rho^2\right) = 0 \tag{2.15}$$

just like a normal Legrange undetermined multiplier. Now consider the constrained minimum $|\mathbf{z}^*|^2 = \rho^2$, which satisfy

$$\left.\frac{\partial C}{\partial z_k}\right|_* + \lambda z_k^* = 0 \tag{2.16}$$

and consider some new proposed change $\mathbf{z}'$

$$z_k' = (1-\epsilon)z_k^*, \qquad 0 < \epsilon \ll 1 \tag{2.17}$$

thus

$$z_k' - z_k^* = -\epsilon z_k^* \tag{2.18}$$

Then because ϵ is as small as we like, the following linear expansion holds

$$\begin{aligned} C(\mathbf{z}+\delta\mathbf{z}') - C(\mathbf{z}+\delta\mathbf{z}^*) &= \sum_k \left.\frac{\partial C}{\partial z_k}\right|_* (\delta z_k' - \delta z_k^*) \\ &= -\epsilon \sum_k \left.\frac{\partial C}{\partial z_k}\right|_* z_k^* \\ &= \lambda\epsilon \sum_k (z_k^*)^2 \\ &= \lambda\epsilon\rho^2 \end{aligned} \tag{2.19}$$

if $\lambda < 0$, then it means the suggested new step can have lower cost, which contradicts with $\mathbf{z}^*$ is the constrained optimum, thus the converse must be true

$$\lambda \geq 0 \tag{2.20}$$

But the point of this is that we want there to be a constraint, so $\lambda = 0$ would be useless for us, then notice that minimising Eq.(41) with respect to δz_k is same as minimising

$$L(\delta\mathbf{z}) = \frac{1}{2}|\mathbf{r} + \mathbf{J}\delta\mathbf{z}|^2 + \frac{1}{2}\lambda|\delta\mathbf{z}|^2 \tag{2.21}$$

but this is just a loss function with input $\mathbf{X} = \mathbf{J}$, true output $\mathbf{y} = \mathbf{r}$, weights $\mathbf{w} = \delta\mathbf{z}$, and a L_2/ridge regularisor with regularisation strength $\lambda > 0$. The point of λ punishes large step/weights $\delta\mathbf{z}$ as it is more likely to reduce the validity of the local linear approximation Eq.(33). Then differentiation and setting to zero yields

$$(\mathbf{J}^T\mathbf{J} + \lambda\mathbf{1})\delta\mathbf{z} = -\mathbf{g} \tag{2.22}$$

which is precisely Levenberg-Marquardt.

So in summary, the uncomfortable of λ comes from the broadness of some implicit underlying Gaussian prior belief regarding the size of each step, which we don't know. Gauss-Newton method is iterative local linear least square, and Levenberg-Marquardt is local linear least square, with a regularisor to punish significant deviation from local linearity. The loss function can also be interpreted as the Legrangian up to an additive constant, with the regularisor as the constraint and the regularisation strength as the undetermined multiplier. Lastly, the well known gradient descent for non linear least square is a crude uncontrolled approximation to the two methods above,

$$\mathbf{J}^T\mathbf{J} + \lambda\mathbf{1} \simeq \alpha^{-1}\mathbf{1} \tag{2.23}$$

however, it is nevertheless perhaps the most simple and useful approximations.

3 Classification

a

We now move onto classifications, which is still about finding out a function that takes unseen input to the correct output, however, output are are discrete classes rather than continuous numbers. For example, our ancestors, a caveman, discovered two things in nature, namely: 1. Brown things that run away, and 2. Green things that do not run away. The first one is known as 'animals' and second one is known as 'plants'. This classification is useful, because if our ancestor encounter a new green thing he haven't seen before, he would know it is very unlikely to jump up and bite him. If he encounters a brown running thing, he would be more careful and have a better chance to survive than his caveman peers that do not make this classification. (see chapter 21 of the book on information theory, inference and learning algorithm by the late Sir David Mackay FRS)

For simplicity, we consider binary classical here where there can only be two classes $y = \{+, -\}$, which stands for animal and plant. The input $\mathbf{x} = (x_1, x_2)$ can have two entries, one describing the greeness of the object, the other one describes how fast it can move on its own (without wind). In this case, the classification is just finding a straight line that divides up the 2D (x_1, x_2) plane into two regions: the animal $(+)$ and the plant $(-)$ region. The equation of a straight line is

$$x_2 = mx_1 + c \tag{3.1}$$

where we emphasis left hand side is x_2 not y, because y is discrete and specifies the region. We can rewrite Eq.(3.1) in a way that is easier to generalise to high dimension by letting

$$m = -\frac{w_1}{w_2}, \qquad c = -\frac{w_0}{w_2} \tag{3.2}$$

then substitute back and rearange give

$$w_0 + w_1 x_1 + w_2 x_2 = 0 \tag{3.3}$$

there are still two free variables, because the equation is one constraint. In vector notation

$$\mathbf{x} \cdot \mathbf{w} = -w_0 = d \tag{3.4}$$

which immediately generalises to three dimensions. If $\mathbf{w}$ is a unit vector, then the $\{\mathbf{x}\}$ satisfying this equation all lies on the same plane, the shortest distance from this plane to origin is $|w_0| = |d|$. $\mathbf{w}$ is orthogonal to the plane, and $\mathbf{x}$ can be decomposed into two orthogonal components, with $\mathbf{x}_\parallel$ lies in the plane

$$\mathbf{x} = \mathbf{w}d + \mathbf{w}_\parallel, \qquad \mathbf{x}_\parallel \cdot \mathbf{w} = 0 \tag{3.5}$$

Eq.(3.4) generalises to high dimensions if we choose to be more descriptive about the data, then the plane becomes a hyperplane, and our task becomes find which way this plane is facing $\mathbf{w}$, and how far (and sign) it is from the origin d. Then if we have new data point $\mathbf{x}'$, we can make a prediction by projection onto normal vector $\mathbf{x}' \cdot \mathbf{w} > d$, then we can assign it to $y = +$, similarly we assign $\mathbf{x}' \cdot \mathbf{w} < d$ to $y = -$. Formally, we can use the sign function (e.g. $\text{sgn}(+8) = +1$, $\text{sgn}(-129) = -1$) to write

$$y_i = \text{sgn}(\mathbf{x}^i \cdot \mathbf{w} + w_0) \tag{3.6}$$

the idea is same as before, we have data pair $\{\mathbf{x}^i, y_i\}$ and try to fit this function to it, with adjustable parameters $\mathbf{w}$ and w_0. This is done by choosing a loss function $L(\mathbf{w}, w_0)$ that tells us how bad the $\mathbf{w}$ are, then minimising it. We first introduce some notation, by absorbing w_0 into $\mathbf{w}$, and the corresponding zero index for $x_0^i = 1$. We also simplified notation a bit using $z_i = \sum_j x_j^i w_j$. We now introduce the 'zero-one' loss, ($\operatorname{sgn}(+16) = +1$, $\operatorname{sgn}(-129) = -1$ is a sign function)

$$l_i = \frac{1}{2}(1 - y_i \operatorname{sgn} z_i) \tag{3.7}$$

it is nothing if y and z has the same sign, and contribute 1 if y and z have different sign. So this loss function just add up the number of predictions with wrong signs. However, it takes into no account of the magnitude of the prediction z_i, hinge loss takes this into account

$$l_i = \max(0, 1 - y_i z_i) \tag{3.8}$$

if y_i and z_i differs in sign, then it contributes $1 + |y_i||z_i|$ to the loss, if they have the same sign, but $|z_i| < 1$, it then contribute $1 - |y_i||z_i|$ to the loss, if $|z_i| \geq 1$, then it does not contribute towards the loss, i.e. the prediction is far away from the boundary and sits comfortably in the correct region, lets introduce one last loss function for now, which is the logistic loss

$$l_i = \ln\bigl(1 + e^{-y_i z_i}\bigr) \tag{3.9}$$

it has the same behaviour as hinge loss for both strongly correct prediction $y_i z_i \gg 0$, where $l_i \to 0$ and strongly wrong prediction $y_i z_i \ll 0$ where $l_i \sim |m|$, but logistic loss is a smoothed out version of the hinge loss. The logistic loss is very widely used in classification, classification with logistic loss is called **logistic regression**. We have started this chapter with classification, now we got back to regression again, lets spend some time to look at the connection between them. Recall from chapter 1 that the error function can be interpreted as a negative log-likelihood (no regularisation has been introduced here yet.), so we associate a probability distribution e^{-l_i}, which is the logistic function

$$P(y_i|z_i) = \frac{1}{1 + e^{-y_i z_i}} = \frac{e^{y_i z_i/2}}{e^{+z_i/2} + e^{-z_i/2}} \tag{3.10}$$

if y_i and z_i has the same sign, then large $|z_i|$ gives high probability that it is in the same class as y_i, for small $|z_i|$, the probability of assigning to one of the two classes are closer to $1/2$, in other words z_i can be interpreted as a propensity. To turn this regression into classification, we simply assign z_i to class $y_i = +$ if $P(+|z_i) > P(-|z_i)$, but this is same as when $z > 0$, thus the model's prediction $\hat{y}_i$

$$\hat{y}_i = \operatorname{sgn} z_i \tag{3.11}$$

which is Eq.(3.6) but with a hat on y_i's head. Thus classification is placing a cut-off on the regression that gives a probability distribution.

b

Now suppose, instead of two classes + and − we have n classes, the natural way to label the this is to use a vector, $y_\alpha = \delta_{\alpha\alpha'}$ for class α. (e.g. for class 2 we have $(y_1 = 0, y_2 =$

$1, y_3 = 0, \ldots, y_n = 0)$), this is called **one-hot encoding**. Then the generalisation of the logistic function Eq.(3.10) is obtained by the replacement $yz/2 \to z_\alpha$

$$P(y_\alpha|z_\alpha) = \frac{e^{z_\alpha}}{\sum_\alpha e^{z_\alpha}} \tag{3.12}$$

where we also suppressed the data label i to reduce clutter. This distribution is called **softmax** and somewhat resembles a Boltzmannn distribution in statistical mechanics where $z_\alpha = -\beta E_\alpha$ where $\beta = 1/k_B T$ and E_α is the energy of stationary state $|\alpha\rangle$ both are obtained by solving the time independent Schrödinger equation $\hat{H}|\alpha\rangle = E_\alpha|\alpha\rangle$. The total likelihood for one data point belong to class α' depends on the full $\mathbf{z}$, so

$$P(\alpha'|\mathbf{x}) = P(y_{\alpha'} = 1|\mathbf{z}) = p_{\alpha'} = \sum_\alpha p_\alpha \delta_{\alpha\alpha'} = \sum_\alpha p_\alpha y_\alpha = \prod_\alpha p_\alpha^{y_\alpha} \tag{3.13}$$

the last equality holds because when $y_\alpha = 0$ it just multiplies the product by 1. The negative log likelihood is then

$$l = -\ln \prod_\alpha p_\alpha^{y_\alpha} = -\sum_\alpha \ln p_\alpha^{y_\alpha} = -\sum_\alpha y_\alpha \ln p_\alpha \equiv S(y, p) \tag{3.14}$$

which is the **cross entropy** between y_α and p_α, and the loss function is known as a cross-entropy loss. For total loss, we sum it over all data points i, $L = \sum_i l_i$. If the two probability distribution are the same $y_\alpha = p_\alpha$, the the cross entropy becomes Shannon entropy

$$S(p) \equiv -\sum_\alpha p_\alpha \ln p_\alpha \tag{3.15}$$

which is thermodynamics entropy as in defined in reversible heat transfer up to a factor of k_B. To understand cross entropy better, we first define relative entropy/Kullback-Lieber divergence

$$D(p \parallel q) \equiv \sum_\alpha p_\alpha \ln \frac{p_\alpha}{q_\alpha} \tag{3.16}$$

one can show

$$D(p \parallel q) \geq 0 \tag{3.17}$$

by considering the inequality $\ln x \leq x - 1$. Clearly, the relative entropy is related to entropy and cross entropy by

$$D(p \parallel q) = \sum_\alpha p_\alpha \ln p_\alpha - \sum_\alpha p_\alpha \ln q_\alpha = S(p, q) - S(p) \tag{3.18}$$

relative entropy measures how close the two probability distributions q_i and p_i are, and vanishes when $q_i = p_i$. If we assume q_i is the true underlying distribution responsible for generating data, minimising the relative entropy with respect to parameters $\mathbf{w}$, is same as minimising the cross entropy, since $S(p)$ is independent of our parameters.

c

We now make a connection to thermodynamics by defining the true probability distribution as Boltzmann/softmax distribution

$$p_\alpha = \frac{1}{Q} e^{-\beta E_\alpha}, \qquad Q = \sum_\alpha e^{-\beta E_\alpha} = e^{-\beta A} \tag{3.19}$$

where we have also defined true Helmholtz free energy A using partition function Q. q_i is typically not accessible because the dimension of the Hamiltonian is too large to be diagonalised for a condensed phase system, so we wish to find the best p_α to approximate it. Then define variational internal energy $U(\mathbf{w})$

$$U(\mathbf{w}) \equiv \sum_\alpha q_\alpha(\mathbf{w}) E_\alpha \tag{3.20}$$

and variational free energy

$$A_\mathbf{w} \equiv U_\mathbf{w} - TS_\mathbf{w} \tag{3.21}$$

then expanding the inequality Eq.(17) and using the thermodynamic definitions, we obtain the Gibbs-Bogoliubov inequality

$$A_\mathbf{w} - A = \frac{1}{\beta} D(q \parallel p) \geq 0 \tag{3.22}$$

and we can find the best probability distribution by minimising the variational free energy $U_\mathbf{w} - TS_\mathbf{w}$ with respect to parameters $\mathbf{w}$. This is typically done variationally by finding a potential that minimises $A_\mathbf{w}$, such as a harmonic potential, which approximates q by a Gaussian function/normal distribution. Thus, minimising variational free energy is same as minimising relative entropy, which is same as minimising cross entropy and maximising negative log-likelihood for Boltzmann/softmax distribution.

d

At the end, we make a connection to information theory. In particular we will look at source coding with symbol codes. Which just means representing letters with strings of bits 0 and 1 in a book, for example $e = 010$, or $z = 0100$. l_α is the length of the code (e.g. $l_5 = l_e = 3$, $l_{26} = l_z = 4$), and the goal is to use the least number of total bits because, it storage these bits with physical devices such as a hard drive costs money. Let 32 be the total number of letters (a-z and space, comma, full stop, ?, ! and $'$). Suppose there is a true underlying distribution p_α which is a ratio of total number of letters (including space) in this book N, and the number of times this letter comes up n_α, so

$$p_\alpha \equiv \frac{n_\alpha}{N} \tag{3.23}$$

for example in Harry Potter and the Philosopher's Stone, $N = 335007$, the number of time 'h' comes up is $n_h = 22546$ giving $p_h = 0.0673$. We now define a second probability distribution q_α, which changes as we change our rule for coding

$$q_\alpha \equiv \frac{1}{Q} 2^{-l_\alpha}, \qquad Q = \sum_\alpha 2^{-l_\alpha} \tag{3.24}$$

turns out for a uniquely decodable code $Q \leq 1$, and we can choose l_α so that $Q = 1$, which we shall do here (easy to do, for example giving each letter 5 bits satisfies this, although this is unlikely to give short overall code) for simplicity, this will not affect the results of the following argument, further detail is discussed in appendix.

Then consider the average length of our codewords in the book

$$\begin{aligned}\langle l \rangle &= \sum_{\alpha} p_\alpha l_\alpha \\ &= -\sum_{\alpha} p_\alpha \log_2 q_\alpha \\ &= S(p, q)\end{aligned} \tag{3.25}$$

where we now have binary cross entropy. So minimising the average code length is same as minimising the cross entropy between p and q, and we save most amount of memory on our hard drive. One can show the cross entropy is at minimum when $q_\alpha = p_\alpha$, which is when $S(p, q) = S(p)$, so the optimal length which we denote by l^* is entropy

$$\langle l^* \rangle = S(p) \tag{3.26}$$

and the length of each optimal code is

$$l^*_\alpha = -\log p_\alpha \equiv s_\alpha \tag{3.27}$$

the optimal code length has a special name known as **information content** s_α, in other words, the optimal length matches the information content of what it is trying to express and using more bits to express the same information just wastes memory, which is physically reasonable. Entropy is the average information content $S = \langle s \rangle$. The difference between not ideal average code length and ideal average code length/entropy is the relative entropy

$$\langle l \rangle - S = D(p \parallel q) \geq 0 \tag{3.28}$$

See appendix for optimal compression for Harry Potter volume 1, chapter 1.

In summary, we have introduced a simple bindary classification problem that differentiates between a plant and animal, which naturally lead to logistic regression. The we generalised the logistic function to the use in multi-class classification, where it becomes a Boltzmann like distribution known as softmax. Where the loss function is a cross entropy loss. Then we show that minimising this cross entropy loss for inferring the underlying distribution is mathematically equivalent to finding the best potential to the true potential that governs thermodynamics properties and also the same as finding the optimal code length when we are trying compress texts by coding it into bits to store in a memory device.

We have introduced the basics of regression and classification, which falls under the domain of supervised learning. In the next chapter, we will start to look at unsupervised learning, which uses data sets with no predefined labels.

Part II
Unsupervised learning

4 Dimensionally reduction

Suppose you are watching a YouTube Video, or playing a video game, but unfortunately your WI-FI is slow, and you are lagging, and you are willing to reduce the resolutions, to stop lagging, but you want to keep the most important information, this chapter is exactly concerned with this. For simplicity, lets just consider one image (one frame of a video), for example, this picture of Dirac in Fig. 1, which is 1107×750 pixels, where k denotes the resolution of the image, higher the k, higher the resolution $k = 750$ recovers the original image. We will discover the exact meaning of k in the rest of this chapter, and explain how to obtain these images. As one can see, for the Dirac picture $k = 40$ gives almost the same amount of detail of the full $k = 750$ picture. We now explain how these images are obtained. We are concerned with linear dimensionally reduction in this chapter.

We first have an input matrix X, for the Dirac picture, X is 1107×750 matrix, the entries of the matrix is between 0 and 1, $X_{ij} = 0$ denotes the pixel at ith row and jth column is black, and $X_{ij} = 1$ denotes a white pixel, the number between 0 and 1 denotes the various shades of gray.

For a general linear operator X, we can form the Hermitian positive semi-definite operator $X^\dagger X$ ($(X^\dagger X)^\dagger = X^\dagger X$ and $\langle\psi|X^\dagger X|\psi\rangle = |X|\psi\rangle|^2 \geq 0$) which has the eigenvalue equation

$$X^\dagger X|v_i\rangle = \sigma_i^2|v_i\rangle \tag{4.1}$$

where $\sigma_i^2 \geq 0$ and real, and $\{|v_i\rangle\}$ form an orthonormal complete set. Let $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0 = \sigma_{r+1} = \sigma_{r+2} = \cdots = r_n$. Where n is the number of eigenvalues of $A^\dagger A$. For $\sigma_i \neq 0$ (i.e. $\sigma_{i\leq r}$), define

$$|u_i\rangle = \frac{A|v_i\rangle}{\sigma_i} \tag{4.2}$$

notice that $|u_i\rangle$ is eigenvector of $XX^\dagger$ with the same eigenvalue σ_i^2 and it also follows that

$$\langle u_i|u_j\rangle = \frac{\langle v_i|X^\dagger X|v_j\rangle}{\sigma_i\sigma_j} = \frac{\sigma_j^2}{\sigma_i\sigma_j}\delta_{ij} = \delta_{ij} \tag{4.3}$$

thus $|u_i\rangle$ are orthonormal but not complete as $|u_{i>r}\rangle = 0$ is undefined. Now right multiple X by the unit operator

$$X = X\sum_{i=1}^{n}|v_i\rangle\langle v_i| = \sum_{i=1}^{r}|u_i\rangle\sigma_i\langle v_i| \tag{4.4}$$

so if X acts on some vector $|\psi\rangle$ that spans the full n-dimensional space, then the output vector $X|\psi\rangle$ only lives in the r-dimensional subspace spanned by $\{u_{i\leq r}\}$. This is by definition the rank of X ($r = \mathrm{rk}X$). We now introduce some jargon, so it is easier to communicate with professionals, $\{\sigma_i\}$ are **singular values**, $\{|v_i\rangle\}$ are **right singular vectors** and $\{|u_i\rangle\}$ are **left singular vectors**. Eq.(4.4) is essentially the result we want to prove, but let's rewrite it in conventional notation. We do this by choosing another

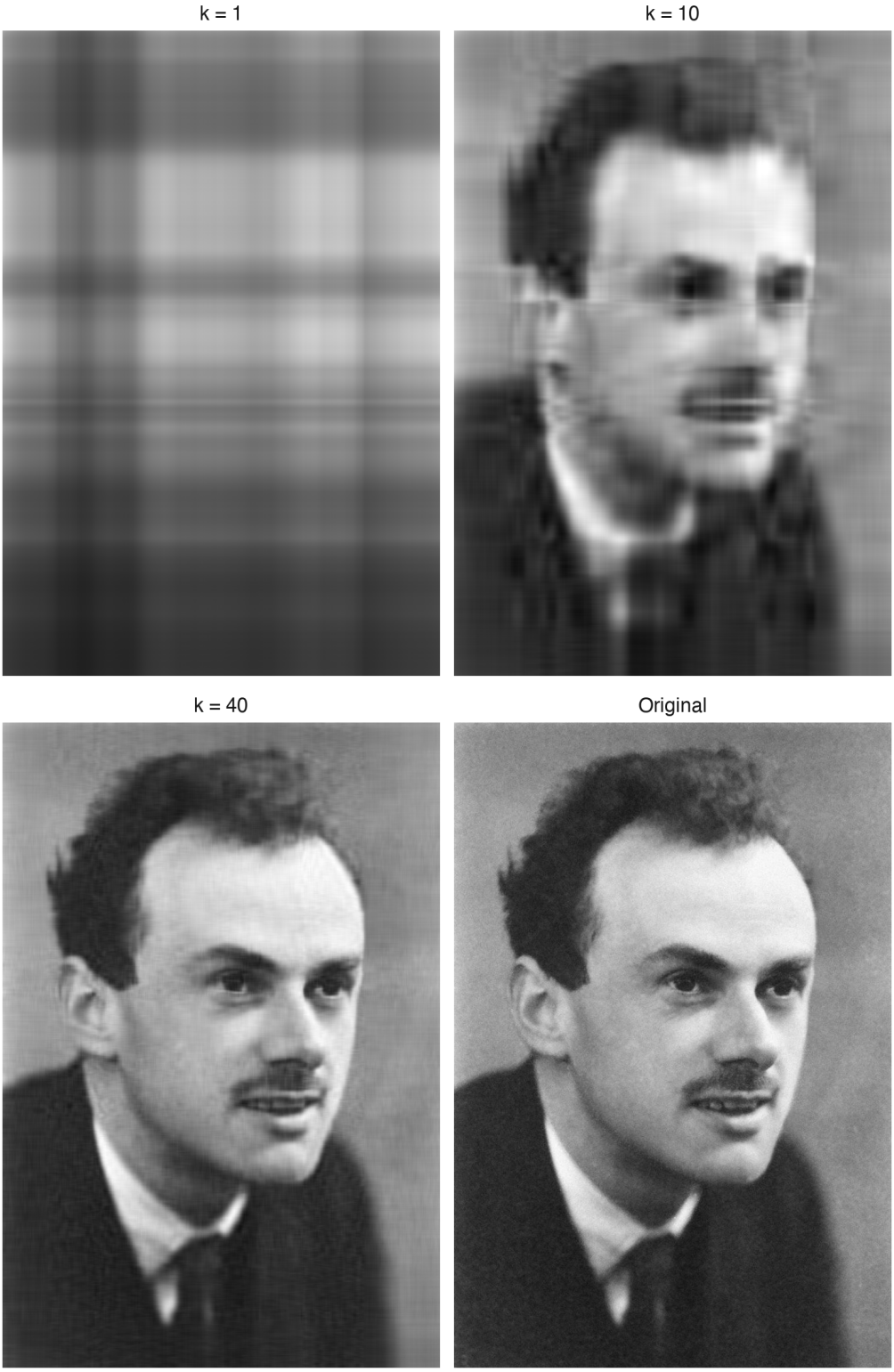


Figure 1: Singular value decomposition of the Dirac photograph: $k = 1$ (top left), $k = 10$ (top right), $k = 40$ (bottom left), and the original (bottom right). Each matrix entry represents grayscale brightness, from 0 (black) to 1 (white).

orthonormal complete set $\{|e_i\rangle\}$, then take an inner product by squeezing the equation between $\langle e_j|$ and $|e_k\rangle$:

$$\langle e_j|X|e_k\rangle = \sum_{i=1}^{r} \langle e_j|u_i\rangle \sigma_i \langle v_i|e_k\rangle \tag{4.5}$$

define matrix element $X_{jk} = \langle e_j|X|e_k\rangle$ then note the overlap between two orthogonal vector is a matrix element of a unitary operator

$$U_{ji} = \langle e_j|u_i\rangle, \qquad V_{ki} = \langle e_k|v_i\rangle \tag{4.6}$$

and $\Lambda_{ij} = \sigma_i \delta_{ij}$ is a matrix (can be rectangular) only with diagonal entries that are non-zero. (find the biggest square matrix in the rectangular matrix, then the square matrix is diagonal) Then we have the same result in conventional notation

$$X = U\Lambda V^\dagger \tag{4.7}$$

Thus any linear transformation can be regarded as a rotation $V^\dagger$, followed by a stretching Λ then followed by another rotation U. This is known as **single value decomposition (SVG)** and it is a very powerful result. If $X^\dagger = X$, then $|u_i\rangle = |v_i\rangle$ and Eq.(4,7) just says a Hermitian matrix can be diagonalised by a unitary transformation, as expected, however singular value decomposition generalises this and applies to any arbitrary rectangular complex matrix.

So now we have factorised the pixel brightness matrix of Dirac photo, we now want to find a lower resolution of it that keeps the essential details. Mathematically, this is same as finding the best low rank approximation $X^{(k)}$ to X, this is in fact another least square problem. But first, lets consider something. A vector's norm (length) is given by the square root of its self inner product $||\psi\rangle|^2 = \langle\psi|\psi\rangle$. Then the natural question is whether matrix also have a 'norm' measuring how big it is compared to other matrix that operates into the same dimension. Turns out, there is, and it is known as **Frobenius norm** square, defined as the matrix self inner product

$$||X\rangle|^2 \equiv \langle X|X\rangle = \sum_{ij} |X_{ij}|^2, \qquad \langle A|B\rangle \equiv \mathrm{tr}\big[A^\dagger B\big] \tag{4.8}$$

There is no confusion with vector inner product because if a matrix sits inside the bracket, then it automatically implies this trace inner product. Then taking the trace in the basis $|v_i\rangle$ gives

$$\langle X|X\rangle = \sum_i \sigma_i^2 \tag{4.9}$$

which is non-negative, good, because length cannot be negative. And X has the same length as a vector with $(\sigma_1, \sigma_2, \cdots, \sigma_n)$ as its entries. So the 'length of matrix' measure how much it stretches all the vectors. Then the best low rank approximation $X^{(k)}$ is the one that minimises the length square of the difference

$$||X - X^{(k)}\rangle|^2 = \langle X - X^{(k)}|X - X^{(k)}\rangle \tag{4.10}$$

in appendix C, we show this minimisation is obtained by

$$X^{(k)} = \sum_{i=1}^{k} |u_i\rangle \sigma_i \langle v_i| \tag{4.11}$$

which just removes the $i > k$ terms X in Eq.(4.4) with progressively smaller σ_is, i.e. the less important details. Turns out, despite that Eq.(4.11) is deduced from the least square problem/Frobenius norm, it applies more generally to other norms.

We will describe **principle component analysis (PCA)**, which is a SVD based linear dimensionally reduction technique. Data is first normalised and the mean is centred on origin,(data is standardised) we show how this is done in appendix E.

PCA can be interpreted as fitting an high dimensional ellipsoid to the data, The eigenvectors of $|v_i\rangle$ are the semi-axis, and the singular values σ_i are the elongation of the ellipsoid in that direction. Thus the idea of fitting not only applies to supervised learning, but also in non-supervised learning.

At the end of this chapter, we make a connection to information theory. Generalisation of Eq.(3.15) for a probability density is

$$S = -\int dx\, p(x) \ln p(x) = -\langle \ln p(x) \rangle \tag{4.12}$$

for a Gaussian distribution

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2} \tag{4.13}$$

then

$$\begin{aligned} S &= \frac{1}{2\sigma^2} \langle (x-\mu) \rangle^2 + \frac{1}{2} \ln\bigl(2\pi\sigma^2\bigr) \\ &= \frac{\sigma^2}{2\sigma^2} + \frac{1}{2} \ln\bigl(2\pi\sigma^2\bigr) \\ &= \ln \sigma + c \end{aligned} \tag{4.14}$$

where we have packed the constants into $c = \frac{1}{2}\ln(2\pi e)$. For r dimensional Gaussian the total entropy is additive, the uncertainty/standard deviation of each direction $|v_i\rangle$ is σ_i therefore

$$S = \sum_{i=1}^{r} \ln \sigma_i + rc \tag{4.15}$$

thus the best rank-k approximation removes the terms/directions with the smallest contribution to entropy, (remember $\sigma_1 \geq \sigma_2 \geq \cdots$) which is the average information content. For a general non-Guassian distribution, the result does not hold, however, it remains heuristically useful.

5 Clustering

Appendix

A Source coding

In Harry Potter and the philosopher's stone, Chapter 1 The Boy Who Lived, there are $N = 25342$ retained characters. The total encoded bit length is

$$L = \sum_{\alpha} n_\alpha l_\alpha = 109003\,\text{bits} \tag{A.1}$$

the average code length per character is

$$\langle l \rangle = \sum_{\alpha} p_\alpha l_\alpha = \frac{\sum_\alpha n_\alpha l_\alpha}{\sum_\alpha n_\alpha} = \frac{L}{N} = 4.301\,\text{bits} \tag{A.2}$$

the relative entropy is

$$D(p \parallel q) = \langle l \rangle - S = 0.0431\,\text{bits} \tag{A.3}$$

and we have

$$Q = \sum_{\alpha=1}^{32} 2^{-l_\alpha} = \frac{2}{2^3} + \frac{7}{2^4} + \frac{4}{2^5} + \frac{10}{2^6} + \frac{3}{2^7} + \frac{3}{2^9} + \frac{1}{2^{10}} + \frac{2}{2^{11}} = 1 \tag{A.4}$$

Symbol	Count	Code	Symbol	Count	Code
a	1,472	0100	q	6	11111111110
b	348	110100	r	1,171	11000
c	403	110101	s	1,293	1001
d	1,047	10110	t	1,688	1010
e	2,483	000	u	635	11001
f	351	110110	v	216	1111101
g	506	110111	w	490	111010
h	1,256	0101	x	28	111111101
i	1,174	0110	y	484	111011
j	24	111111100	z	10	11111111111
k	224	1111100	Space	4,532	001
l	988	10111	.	417	111100
m	494	111000	,	290	111101
n	1,249	0111	?	41	111111110
o	1,470	1000	!	18	1111111110
p	364	111001	’	170	1111110

Table 1: Character counts and an optimal binary Huffman code for the filtered first chapter of *Harry Potter and the Philosopher's Stone*

Encoding	Total bits	Bits per character
Fixed-length, 32 symbols	126,710	5
Huffman	109,003	4.3013
Ideal entropy benchmark $NH_2(p)$	107,909.84	4.2581

Table 2: Encoding costs for the 25,342 retained characters, excluding codebook and storage-padding overhead.

B Cholesky factorisation

In this appendix, we wish to prove any positive semi-definite Hermitian matrix H can be written as a product of a lower diagonal matrix $A^\dagger$ and its' Hermitian adjoint A, which is an upper diagonal matrix (used in one of the subroutines for fitting eco frequencies)

$$H = A^\dagger A \tag{B.1}$$

which seems familiar to factorising the quantum harmonic oscillator Hamiltonian (minus a zero point energy) into its ladder operators, where A is the annihilation operator. H is Hermitian, so it has the following spectral expansion

$$H = \sum_i |i\rangle E_i \langle i| \tag{B.2}$$

trivially, there exist $B = \sum_i |i\rangle \sqrt{E_i} \langle i|$, ($\sqrt{E_i}$ is real since $E_i = \langle i|H|i\rangle \geq 0$) such that $B^\dagger B = H$, in fact B is also Hermitian. B is diagonal in the eigenbasis of H, but needs not to be diagonal in an arbitrary basis $\{|e_i\rangle\}$. We want to find the unitary transformation of B so it becomes an upper diagonal matrix. Define

$$|b_j\rangle = B|e_j\rangle \tag{B.3}$$

then construct an orthonormal basis $\{|u_i\rangle\}$ by removing the components of $|b_i\rangle$ that sticks into the subspace spanned by the previous vectors, for the first vector, we just need to normalise $|u_1\rangle = |b_1\rangle / \sqrt{\langle b_1|b_1\rangle}$ then by induction

$$|u'_j\rangle = |b_j\rangle - \sum_{i<j} |u_i\rangle \langle u_i|b_j\rangle \tag{B.4}$$

normalise $|u_j\rangle = |u'_j\rangle / \sqrt{\langle u'_j|u'_j\rangle}$, if $|u'_j\rangle = 0$, choose a unit vector orthogonal to all previous vector, then we can construct a unitary operator

$$U = \sum_i |u_i\rangle \langle e_i| \tag{B.5}$$

then define

$$A = U^\dagger B \tag{B.6}$$

the matrix element in the original basis is

$$A_{ij} = \langle e_i|A|e_j\rangle = \sum_k \langle e_i|e_k\rangle \langle u_k|B|e_j\rangle = \sum_k \langle e_i|e_k\rangle \langle u_k|b_j\rangle = \langle u_i|b_j\rangle \tag{B.7}$$

but $\langle u_i|b_j\rangle = 0$ for $i > j$, thus $A_{ij} = 0$ for $i > j$, which is same as saying A is upper triangular, and finally

$$A^\dagger A = B^\dagger U U^\dagger B = B^\dagger B = H \tag{B.8}$$

C Young-Eckart theorem

The rank-k approximation to A, which is $A^{(k)}$ minimises the norm $\langle A - A^{(k)}|A - A^{(k)}\rangle$. The single value decomposition of $A^{(k)}$ is analogous of A

$$A = \sum_{i=1}^{r} |u_i\rangle\sigma_i\langle v_i|, \qquad A^{(k)} = \sum_{j=1}^{k} |u_j^{(k)}\rangle\sigma_j^{(k)}\langle v_j^{(k)}| \tag{C.1}$$

then

$$\langle A - A^{(k)}|A - A^{(k)}\rangle = \langle A|A\rangle + \langle A^{(k)}|A^{(k)}\rangle - 2\mathrm{Re}\langle A|A^{(k)}\rangle \tag{C.2}$$

Order the singular values so that

$$\sigma_1 \geq \cdots \geq \sigma_r \geq 0, \qquad \sigma_1^{(k)} \geq \cdots \geq \sigma_k^{(k)} \geq 0.$$

By von Neumann's trace inequality (see Appendix D),

$$|\langle A|A^{(k)}\rangle| \leq \sum_{i=1}^{k} \sigma_i\sigma_i^{(k)}. \tag{C.3}$$

then also note $\mathrm{Re}\langle A|A^{(k)}\rangle \leq |\langle A|A^{(k)}|$. Substituting this into Eq.(C.2) gives

$$\begin{aligned}
\langle A - A^{(k)}|A - A^{(k)}\rangle &\geq \sum_{i=1}^{r} \sigma_i^2 + \sum_{j=1}^{k} \sigma_j^{(k)2} - 2\sum_{i=1}^{k} \sigma_i\sigma_i^{(k)} \\
&= \sum_{i=k+1}^{r} \sigma_i^2 + \sum_{i=1}^{k} (\sigma_i^2 + \sigma_i^{(k)2} - 2\sigma_i\sigma_i^{(k)}) \\
&= \sum_{i=k+1}^{r} \sigma_i^2 + \sum_{i=1}^{k} (\sigma_i - \sigma_i^{(k)})^2 \\
&\geq \sum_{i=k+1}^{r} \sigma_i^2
\end{aligned} \tag{C.4}$$

thus the best possible low rank approximation $A^{(k)}$ should saturate the bound. Consider

$$A^{(k)} = \sum_{i=1}^{k} |u_i\rangle\sigma_i\langle v_i| \tag{C.5}$$

then

$$A - A^{(k)} = \sum_{i=1}^{r} |u_i\rangle\sigma_i\langle v_i| - \sum_{i=1}^{k} |u_i\rangle\sigma_i\langle v_i| = \sum_{i=k+1}^{r} |u_i\rangle\sigma_i\langle v_i| \tag{C.6}$$

so

$$\begin{aligned}\langle A - A^{(k)}|A - A^{(k)}\rangle &= \mathrm{tr}\left[\sum_{i=k+1}^{r} |v_i\rangle\sigma_i\langle u_i| \sum_{j=k+1}^{r} |u_j\rangle\sigma_j\langle v_j|\right] \\ &= \sum_{i=k+1}^{r}\sum_{j=k+1}^{r} \sigma_i\sigma_j\langle u_i|u_j\rangle\langle v_j|v_i\rangle \\ &= \sum_{i=k+1}^{r}\sum_{j=k+1}^{r} \sigma_i\sigma_j\langle u_i|u_j\rangle\delta_{ij} \\ &= \sum_{i=k+1}^{r} \sigma_i^2 \end{aligned} \tag{C.7}$$

which saturates the bound Eq.(C.9) thus $A^{(k)}$ in Eq.(C.10) is indeed the best low rank approximation to A.

D Von Neumann trace inequality

For two operator A and B, with following spectral expansion of $A^\dagger A$ and $B^\dagger B$

$$A^\dagger A = \sum_i |a_i\rangle \alpha_i^2 \langle a_i|, \qquad B^\dagger B = \sum_j |b_j\rangle \beta_j^2 \langle b_j| \tag{D.1}$$

where $\alpha_1 \geq \alpha_2 \geq \cdots$, $\beta_1 \geq \beta_2 \geq \cdots$, then define

$$|a_i'\rangle = \frac{A|a_i\rangle}{\alpha_i}, \qquad |b_j'\rangle = \frac{B|b_j\rangle}{\beta_j} \tag{D.2}$$

we have single value decompositions

$$A = \sum_i |a_i'\rangle \alpha_i \langle a_i|, \qquad B = \sum_j |b_j'\rangle \beta_j \langle b_j| \tag{D.3}$$

then

$$\begin{aligned} |\langle A|B\rangle| &= \left| \mathrm{tr} \sum_{ij} |a_i\rangle \alpha_i \langle a_i'|b_j'\rangle \beta_j \langle b_j| \right| \\ &= \left| \sum_{ij} \alpha_i \beta_j \langle a_i'|b_j'\rangle \langle b_j|a_i\rangle \right| \\ &\leq \sum_{ij} \alpha_i \beta_j \left| \langle a_i'|b_j'\rangle \langle b_j|a_i\rangle \right| \end{aligned} \tag{D.4}$$

define

$$c_{ij} = \frac{1}{2}\left(|\langle a_i'|b_j'\rangle|^2 + |\langle a_i|b_j\rangle|^2 \right) \tag{D.5}$$

then use

$$0 \leq (|x| - |y|)^2 = |x|^2 + |y|^2 - 2|x||y| \tag{D.6}$$

thus

$$2|xy| \leq |x|^2 + |y|^2 \tag{D.7}$$

we have

$$|\langle A|B\rangle| \leq \sum_{ij} \alpha_i \beta_j c_{ij} \tag{D.8}$$

also $0 \leq c_{ij} \leq 1$ and

$$\sum_i c_{ij} = \sum_j c_{ij} = 1 \tag{D.9}$$

We now transform c_{ij} into δ_{ij} without decreasing the weighted sum $\sum_{ij} \alpha_i \beta_j c_{ij}$. If $c_{11} < 1$, there must be $i, j > 1$ with $c_{i1} > 0$ and $c_{1j} > 0$. Set

$$\varepsilon = \min(c_{i1}, c_{1j}), \tag{D.10}$$

and change these four entries:

$$\begin{pmatrix} c_{11} & c_{1j} \\ c_{i1} & c_{ij} \end{pmatrix} \longrightarrow \begin{pmatrix} c_{11} + \varepsilon & c_{1j} - \varepsilon \\ c_{i1} - \varepsilon & c_{ij} + \varepsilon \end{pmatrix}. \tag{D.11}$$

This preserves nonnegativity and all row and column sums. The weighted sum changes by

$$\begin{aligned}&\varepsilon\left(\alpha_1\beta_1+\alpha_i\beta_j-\alpha_1\beta_j-\alpha_i\beta_1\right)\\&\quad=\varepsilon(\alpha_1-\alpha_i)(\beta_1-\beta_j)\geq 0.\end{aligned}\tag{D.12}$$

Each move eliminates at least one nonzero off-diagonal entry from the first row or column. Repeating gives $c_{11}=1$, with all other entries in that row and column zero. Repeat the argument on the remaining rows and columns, eventually obtaining $c_{ij}=\delta_{ij}$. Therefore,

$$\sum_{ij}\alpha_i\beta_j c_{ij}\leq\sum_{ij}\alpha_i\beta_j\delta_{ij}=\sum_i\alpha_i\beta_i.\tag{D.13}$$

Combining this with the preceding bound Eq.(D.8) gives

$$|\langle A|B\rangle|\leq\sum_i\alpha_i\beta_i.\tag{D.14}$$

E Data standardisation

Let $\tilde{X}$ denotes original data on a $n \times d$ grid and define $|I\rangle = (1, 1, 1, \cdots)^T$ (n entries), then $\langle I|\tilde{X}/n$ is a row vector with ith entry as the average of column i of $\tilde{X}$, therefore the centred data is

$$X_c = \tilde{X} - \frac{1}{n}|I\rangle\langle I|\tilde{X} \equiv \tilde{X} - \mu \tag{E.1}$$

then introduce

$$N = \sqrt{n}\sum_{i=1}^{d} |e_i\rangle\langle e_i|X_c^\dagger X_c|e_i\rangle^{-1/2}\langle e_i| \equiv \sum_{i=1}^{d} |e_i\rangle\frac{1}{s_i}\langle e_i| \tag{E.2}$$

where $s_i = 0$ are exluded from the sum to prevent division by zero, then the normalised and centred data can be written as

$$X = X_c N \tag{E.3}$$

We show that the average of ith column of X is

$$\begin{aligned}
\frac{1}{n}\langle I|X|e_i\rangle &= \frac{1}{n}\langle I|X_c N|e_i\rangle \\
&= \frac{1}{ns_i}\langle I|X_c|e_i\rangle \\
&= \frac{1}{ns_i}\langle I|\left(1 - \frac{1}{n}|I\rangle\langle I|\right)\tilde{X}|e_i\rangle \\
&= \frac{1}{ns_i}\left(1 - \frac{\langle I|I\rangle}{n}\right)\langle I|\tilde{X}|e_i\rangle
\end{aligned} \tag{E.4}$$

but notice that

$$\langle I|I\rangle = n \tag{E.5}$$

thus

$$\frac{1}{n}\langle I|X|e_i\rangle = 0 \tag{E.6}$$

as desired. Then the variance of row i is

$$\begin{aligned}
\frac{1}{n}\langle e_i|X^\dagger X|e_i\rangle &= \frac{1}{ns_i^2}\langle e_i|X_c X_c|e_i\rangle \\
&= \frac{ns_i^2}{ns_i^2} \\
&= 1
\end{aligned} \tag{E.7}$$

as desired.